

SC2006 Software Engineering

Lecture 23 Supplement

Conrad Watt

Plan for the rest of the semester

- Thursday 17th April
 - non-assessed case study lecture

Past paper - 2022-2023 Sem 2

Q1b

Use case description - request replacement

Initiating actor: User

Other actors: Company

Pre-condition:

Request refers to a valid order

Inputs: order id

Past paper - 2022-2023 Sem 2

Q1b

1. assert: the order was delivered within the past 7 days
2. the chatbot requests that the user provides item pictures, receipt of purchase, and reason for refund
3. the information is sent to the company
4. if the company accepts the refund, the chatbot displays the replacement procedures to the customer

Past paper - 2022-2023 Sem 2

Q1b

AF-1: the order is not delivered in the last 7 days

1. the chatbot explains that the refund is not allowed
2. end

AF-2: the user does not provide the requested information correctly

1. the chatbot displays an error message
2. go back to main step 1

AF-4: the company denies the refund

1. the chatbot displays the company's reason for denial
2. end

Past paper - 2022-2023 Sem 2

Q2a

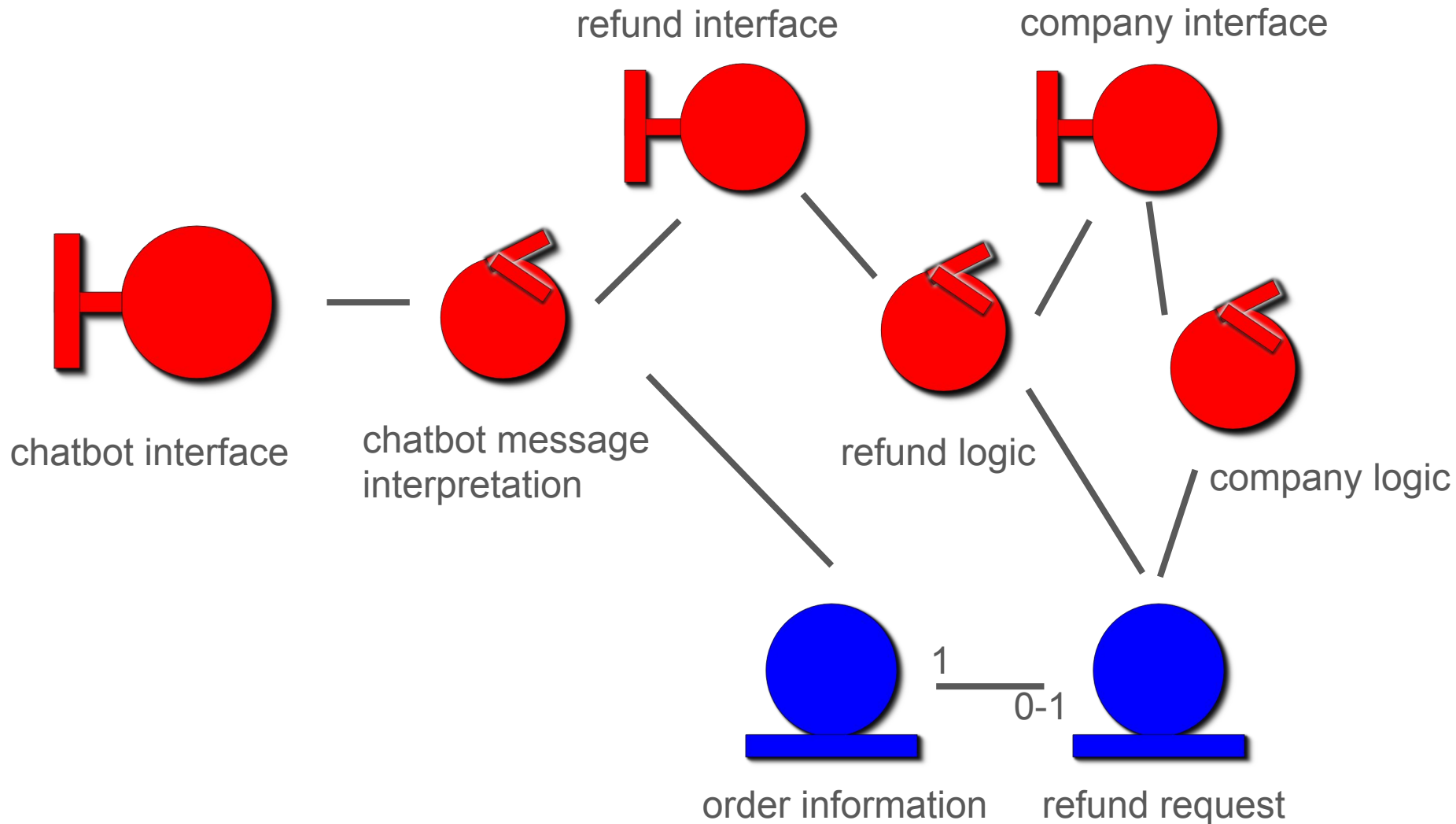
main classes:

boundary: chatbot interface, company interface,
refund interface(?)

control: chatbot message interpretation, refund logic,
company logic

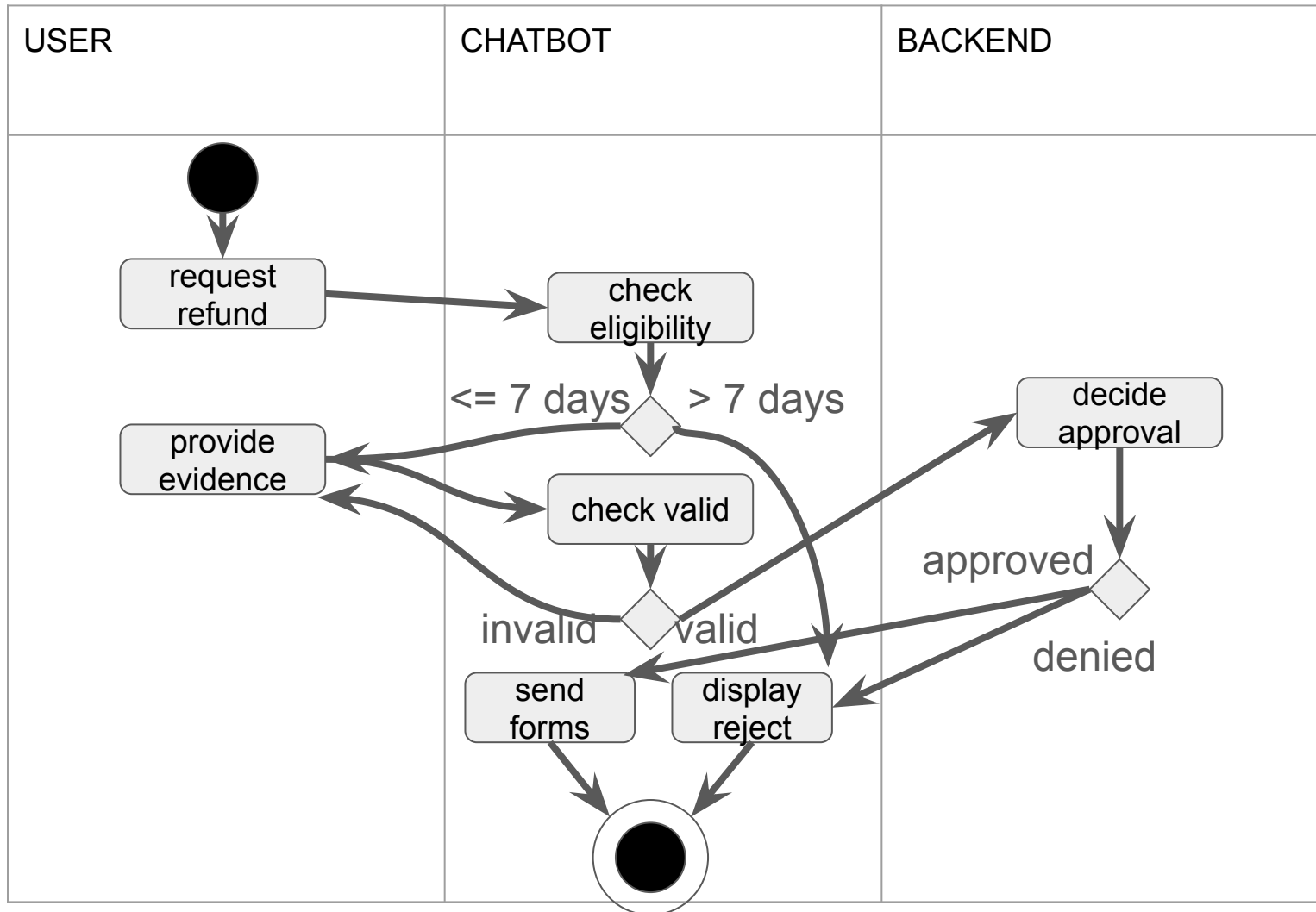
entity: order information, refund request

Past paper - 2022-2023 Sem 2



Past paper - 2022-2023 Sem 2

Q2b

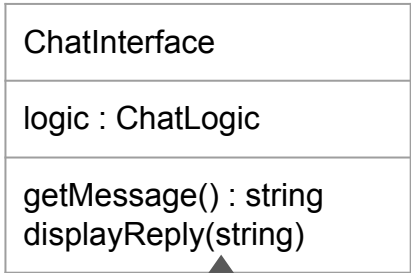


Past paper - 2022-2023 Sem 2

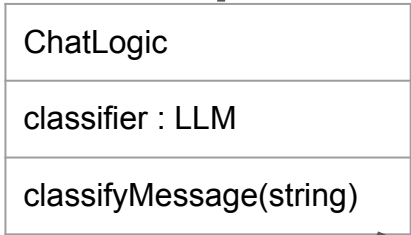
Q3ai my architecture - a 4 layer architecture

1. interface
2. message classification
3. functionality
4. state

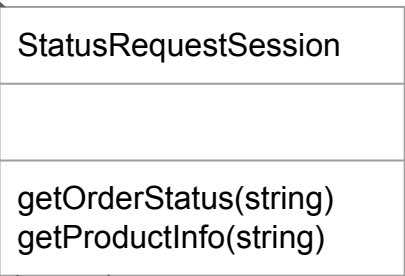
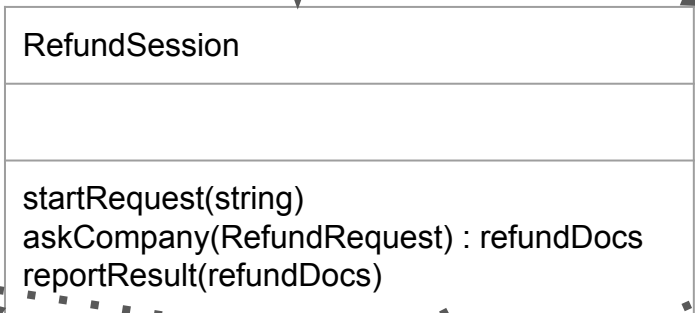
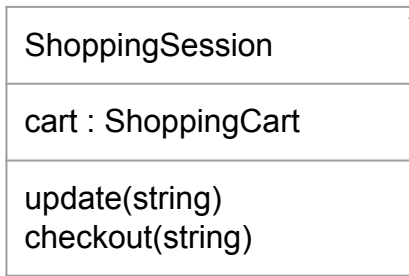
UI



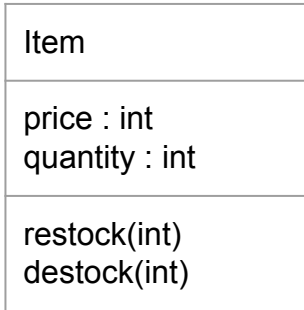
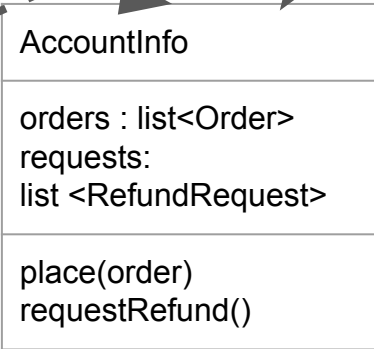
Chat engine



Request handlers



State



Case Study - Horizon Post Office Scandal

- The Post Office (UK's national post service) contracts company F to develop new accounting software for use by its franchisees
- at the time, the largest non-military IT contract in Europe
- the concept - keep an electronic list of all transactions to check periodically against the actual money accounts of the franchisee

Case Study - Horizon Post Office Scandal

- The accounting software is poor quality and riddled with bugs
 - many franchisees are recorded by the software as having amounts of cash on hand that don't match their physical records
 - in normal circumstances this would indicate fraud
- On the basis of these records, the Post Office prosecutes over 900 franchisees for fraud, leading to severe fines, confiscations, and jail time

Case Study - Horizon Post Office Scandal



Case Study - Horizon Post Office Scandal

- Media reporting eventually uncovers the scale of the injustice
 - https://www.private-eye.co.uk/pictures/special_reports/justice-lost-in-the-post.pdf
 - <https://www.computerweekly.com/feature/Post-Office-Horizon-scandal-explained-everything-you-need-to-know>
- Many of the initial financial discrepancies came from software bugs
 - social and political reasons that the scandal grew so large - not the subject of this talk
- This talk - the kind of bugs that caused the discrepancies, and why it's important to build good software

Case Study - Horizon Post Office Scandal

- All of these bugs are drawn from official court documents related to a recent case where post office workers sued the post office for compensation regarding their wrongful convictions
 - <https://www.judiciary.uk/wp-content/uploads/2022/07/bates-v-post-office-appendix-1-1.pdf>
 - £128m in compensation paid out as of January 2025

Case Study - Horizon Post Office Scandal

- Anonymous former software engineer at company F:
 - <https://archive.ph/5Inru>
 - “no design documents, no test documents, no peer reviews, no code reviews, no coding standards”
 - “to my knowledge, no one on the team had a computer science degree or any degree-level qualifications in the right field”

Case Study - Horizon Post Office Scandal

- Bug 1:
 - “In April 2003... Horizon version S30 was released by [Company F] and this introduced a bug where the value of transactions reversed by Subpostmasters was shown twice in the amount of the reversal in branch accounts.”
 - “the issue was caused by a software error... which incorrectly calculated the reversal value and quantity (essentially, the wrong mathematical sign was applied when reversing... transactions (+ instead of -))”
- So if a postmaster recorded that they had received an incoming transaction, and then later reversed it, the system would **increase** their recorded “cash on hand” rather than reducing it

Case Study - Horizon Post Office Scandal

- Bug 1:
 - typing “+” instead of “-” when coding the arithmetic to reverse a transaction is a trivial error
 - this kind of bug should be caught by even rudimentary unit tests
 - failing to do these tests can have devastating consequences!

Case Study - Horizon Post Office Scandal

- Bug 2:
 - Essentially, if a user input a transaction (e.g. recording incoming cash) into their terminal and it failed, they might try to input the transaction on another terminal instead. But then when the first terminal is restarted, the failed transaction is silently re-attempted.
 - “[The bug] had the effect of preventing a counter from [recording transactions]. This was not always immediately obvious to the user of the counter. This could result in them thinking that some transactions which had been entered, were missing, and so they attempted to re-enter the transactions on another counter. When the offending counter was re-started, both versions of the transaction become visible and this could cause errors in the accounts.”

Case Study - Horizon Post Office Scandal

- Bug 2:
 - Transactions are hard! But it's often better to be "fail-fast" - if there's an error, stop everything rather than trying to silently re-attempt the transaction sometime later
 - Imagine doing online shopping and your first credit card fails, so you restart the purchase with a second card. But then the next day the website silently tries to "replay" your purchase with the first card even though your second attempt succeeded.

Case Study - Horizon Post Office Scandal

- Bug 3:
 - Under certain circumstances where a crash occurs in the middle of a transaction, the user could be left with an apparently frozen screen displaying an “enter” button. Each time the button is pressed, part of the last transaction before the crash silently repeats itself
 - “This problem involved issues relating to two different scripts, the Post Log On script and the Forced Log Out script. The latter did not close down the former correctly (or in software terms, did not lead to the former script correctly closing down the operation) which meant that the script was left on the stack of incomplete processes. If the SPM then logged back into the counter to perform a transfer of cash from their core branch to the outreach branch, the “pouch delivery” screen would still show, with “enter” being enabled. If the SPM were then to press “enter”, then the value of the transaction would repeat.”

Case Study - Horizon Post Office Scandal

- Bug 3:
 - Error recovery is hard! But the “fail-fast” principle still applies. If there is a crash, every effort should be made to properly clear/cancel any incomplete operations prior to the crash from the local state
 - UIs should always be robustly coded so that there is visual feedback whenever an important event is triggered
- Lots and lots of other bugs in the court document...

Case Study - Horizon Post Office Scandal

- Bad practices in security / permissions management compounded the issues
- Company F on occasion directly accessed user accounts to correct issues originally caused by bugs
 - No clear record was kept of these interventions
 - Post Office originally reported that “no functionality in Horizon for either a branch, Post Office or [Company F] to edit, manipulate or remove transaction data” - not true!
- This is now a major issue in the legal battles surrounding the scandal - Lesson: don't play fast and loose with sensitive customer/financial information

Case Study - Horizon Post Office Scandal

- Software quality issues are bad for your company in a lot of ways, but in the extreme case they can become legal issues
- There's now discussion of prosecutions and jail time for those involved in covering up the scandal

Case Study - Horizon Post Office Scandal

- Many opportunities to avoid this:
 - proper software engineering practices to prevent many of the bugs
 - Company F could have been more candid about quality of the software
 - the Post Office could have decided not to prosecute so aggressively

Case Study - Horizon Post Office Scandal

- What can you do about this?

Case Study - Horizon Post Office Scandal

- What can you do about this?
- Individual
 - take pride in the quality of your code
 - ensure you're up to date with best practices
 - hold your juniors to a high standard

Case Study - Horizon Post Office Scandal

- What can you do about this?
- Individual
 - take pride in the quality of your code
 - ensure you're up to date with best practices
 - hold your juniors to a high standard
- Team
 - Enforce standards for systems design, code review, and testing
 - Have a robust process for monitoring and correcting bugs
 - Protect members from company pressures to prematurely ship

Case Study - Horizon Post Office Scandal

- What can you do about this?
- Individual
 - take pride in the quality of your code
 - ensure you're up to date with best practices
 - hold your juniors to a high standard
- Team
 - Enforce standards for systems design, code review, and testing
 - Have a robust process for monitoring and correcting bugs
 - Protect members from company pressures to prematurely ship
- Company
 - Keep accurate view on product quality and development progress
 - Ensure the team is adequately resourced for quality control
 - Don't rush products out in an insecure / dangerous state

Case Study - Horizon Post Office Scandal

```
function reverseSign(d) {  
    if d < 0 {  
        d = abs(d);  
    } else  
        d = d - (d*2);  
    }  
    return d;  
}
```

Whole function is equivalent to the single line `d = -d`

“Whoever wrote this code clearly has no understanding of elementary mathematics or the most basic rules of programming”

<https://www.postofficehorizoninquiry.org.uk/sites/default/files/2022-11/FUJ00080690%20Report%20on%20the%20EPOSS%20PinICL%20Task%20Force%2014052001.pdf>

Shameless advert

Next semester I'm starting a third year MPE/BDE course (still in its final stages of approval)

SC3270 - Reasoning about Programs

Testing *isn't good enough*! Learn how to really **prove** that a program is behaving correctly(*) under all circumstances! Plus, learn how to prove that a programming language's type system protects it from errors(*)!

(*) we will spend a decent part of the course agonising over the precise definition of “correctly” and “errors” :)

Prerequisites: SC2001 or SC2301 or (MH1812 and MH1403)